

Bond-Angle Distribution Module Specification

Species-Resolved Three-Body Geometry for AtomisticFrameCollection Analysis

mdstats

2026-07-11

1. Purpose

This document specifies the public bond-angle distribution module

`mdstats/analysis/bond_angle.py`

The module evaluates species-resolved local three-body geometry in an `AtomisticFrameCollection`. It is valid for:

- time-ordered trajectories;
- independent structural ensembles;
- single static structures.

The module does not require velocities or a time axis. Each frame is analyzed independently and then reduced according to an explicit averaging convention.

The intended workflow is

```
pair RDF
|
v
first-shell minimum
|
v
PairCutoffRegistry
|
v
shared neighbor lists
|
v
coordination filters
|
v
A-B-C bond-angle distribution
```

The term *bond angle* denotes an angle constructed from a fixed neighbor rule. For liquids or weakly associated species, *neighbor-angle distribution* may be the more precise physical description.

2. Theory and triplet convention

For triplet

$$A - B - C,$$

B is always the central species. Let atom $j \in B$ be the center, atom $i \in A$ an A neighbor, and atom $k \in C$ a C neighbor. Using minimum-image vectors from the center,

$$\mathbf{u}_{ji} = \mathbf{r}_i - \mathbf{r}_j, \quad \mathbf{v}_{jk} = \mathbf{r}_k - \mathbf{r}_j,$$

the angle is

$$\theta_{ijk} = \cos^{-1} \left[\frac{\mathbf{u}_{ji} \cdot \mathbf{v}_{jk}}{\|\mathbf{u}_{ji}\| \|\mathbf{v}_{jk}\|} \right].$$

The cosine is clipped to $[-1, 1]$ before applying `arccos`.

The endpoint neighbor sets are

$$\mathcal{N}_A(j) = \{i \in A : r_{ij} < r_{AB}^{\text{cut}}\},$$

and

$$\mathcal{N}_C(j) = \{k \in C : r_{jk} < r_{BC}^{\text{cut}}\}.$$

Both cutoffs come from one `PairCutoffRegistry`.

2.1 Symmetric endpoint species

If $A = C$, each physical angle is counted once by using unordered endpoint pairs:

$$i < k, \quad i, k \in \mathcal{N}_A(j).$$

For central coordination q , the number of possible angles is

$$\binom{q}{2} = \frac{q(q-1)}{2}.$$

2.2 Asymmetric endpoint species

If $A \neq C$, endpoint roles are distinct. The angle set is the Cartesian product

$$\mathcal{N}_A(j) \times \mathcal{N}_C(j),$$

with count

$$q_A(j)q_C(j).$$

3. Neighborhood definition

The module never chooses cutoffs automatically. The user first calculates an RDF, evaluates an appropriate first-shell minimum, and constructs fixed cutoffs. For example:

```

rdf_si_o = compute_pair_rdf(
    collection,
    species_a="Si",
    species_b="O",
    r_max=4.0,
)

si_o_cutoff = PairCutoff.from_rdf_minimum(rdf_si_o)

cutoffs = PairCutoffRegistry.from_cutoffs([si_o_cutoff])

```

For an asymmetric triplet:

```

cutoffs = PairCutoffRegistry.from_cutoffs(
    [
        PairCutoff.from_rdf_minimum(rdf_si_o),
        PairCutoff.from_rdf_minimum(rdf_al_o),
    ]
)

```

The registry must contain every pair needed by:

- triplet construction;
- species-specific coordination filters;
- combined-species coordination filters.

A fixed cutoff is applied across all selected frames. Frame-dependent cutoff re-estimation is deliberately excluded because it can turn thermal distance fluctuations into artificial changes of neighborhood definition.

4. Public coordination-filter object

4.1 CoordinationCondition

```

@dataclass(frozen=True, slots=True)
class CoordinationCondition:
    neighbor_species: tuple[int, ...]
    minimum: int | None = None
    maximum: int | None = None

```

`neighbor_species` may contain one or several species. The condition counts the set union of neighbors belonging to those species:

$$N_S(j) = \left| \bigcup_{X \in S} \mathcal{N}_X(j) \right|.$$

Required constraints:

- at least one species is supplied;
- duplicate species are removed or rejected;
- `minimum` and `maximum` are nonnegative integers when present;
- at least one bound is present;
- `minimum` $\leq$ `maximum` when both exist.

Convenience constructors:

```

CoordinationCondition.exact("O", 4)
CoordinationCondition.at_least("O", 5)

```

```
CoordinationCondition.at_most(("Si", "Al"), 2)
CoordinationCondition.between(("Si", "Al"), 1, 2)
```

Examples:

```
# Tetrahedral framework Si only.
CoordinationCondition.exact("O", 4)

# Bridging oxygen with two total framework-cation neighbors.
CoordinationCondition.exact(("Si", "Al"), 2)

# Six-coordinate Na with no more than one nearby Cl.
[
    CoordinationCondition.exact("O", 6),
    CoordinationCondition.at_most("Cl", 1),
]
```

All conditions are applied to the central *B* atom. A center contributes only when every condition passes.

4.2 Implemented helpers

```
condition.symbols
condition.accepts(coordination)
```

`symbols` returns the canonical chemical-symbol tuple. `accepts()` evaluates the inclusive lower and upper bounds after the combined-species neighbor union has been counted.

5. Public API

```
def compute_bond_angle_distribution(
    collection: AtomisticFrameCollection,
    *,
    triplet: tuple[SpeciesLike, SpeciesLike, SpeciesLike],
    cutoffs: PairCutoffRegistry | Mapping[PairKeyLike, float | PairCutoff],
    coordination_filters: Sequence[CoordinationCondition] | None = None,
    bins: int | ArrayLike = 180,
    angle_range: tuple[float, float] = (0.0, 180.0),
    averaging: Literal[
        "angle_weighted",
        "center_weighted",
        "frame_weighted",
    ] = "angle_weighted",
    frame_start: int | None = None,
    frame_stop: int | None = None,
    frame_step: int = 1,
    center_atom_indices: ArrayLike | None = None,
    endpoint_a_atom_indices: ArrayLike | None = None,
    endpoint_c_atom_indices: ArrayLike | None = None,
    per_frame: bool = False,
    return_angles: bool = False,
    block_size: int = 256,
    neighbor_search_options: NeighborSearchOptions | None = None,
) -> BondAngleDistributionResult:
    ...
```

5.1 Parameters

collection

Normalized `AtomisticFrameCollection`. Required fields are atomic numbers, cell matrices, PBC flags, and positions for every frame.

triplet

Three species in endpoint-center-endpoint order:

```
("O", "Si", "O")
("Si", "O", "Al")
("O", "Na", "O")
```

The center is always the second species.

cutoffs

A `PairCutoffRegistry` or a mapping coercible to one. The registry must contain pairs $A - B$ and $B - C$, plus every center-filter species pair.

coordination_filters

Optional sequence of central-atom filters. All conditions must pass. An empty sequence is equivalent to `None`.

bins

Either:

- integer number of equal-width bins; or
- strictly increasing angle-bin edges in degrees.

angle_range

For integer `bins`, this defines the equal-width histogram range. For explicit bin edges, the implementation still validates `angle_range` and requires all edges to lie inside it. The accepted range must be finite and satisfy

$$0^\circ \leq \theta_{\min} < \theta_{\max} \leq 180^\circ.$$

averaging

Selects the default distribution returned by the `distribution` property. All three averaging views are retained in the result.

Frame selection

`frame_start`, `frame_stop`, and `frame_step` use ordinary Python slicing semantics. At least one frame must be selected and `frame_step` > 0 .

Explicit atom indices

Optional canonical atom-index restrictions for the center and endpoint groups. When species labels and indices are both supplied, every index must belong to the corresponding species. Duplicate indices are rejected. When $A = C$, the resolved A- and C-endpoint selections must be identical; otherwise the symmetric unordered-pair definition would be ambiguous.

per_frame

When **True**, retain per-frame counts, densities, angle counts, accepted-center counts, and validity flags.

return_angles

When **True**, retain every accepted angle in degrees. This can consume large memory and is disabled by default.

block_size

Center-block size used by the dense oracle.

neighbor_search_options

Optional **NeighborSearchOptions**. One analysis-local executor serves endpoint and central-filter requests across all selected frames. Automatic Verlet reuse is limited to eligible time-ordered trajectories; single-frame selections and independent ensembles are stateless unless an expert explicitly requests caching. Each distinct request keeps its own digest and candidate cache. Angle values and weighting are backend-neutral.

6. Result object

```
@dataclass(slots=True)
class BondAngleDistributionResult:
    triplet: tuple[str, str, str]

    bin_edges: NDArray[np.float64]
    bin_centers: NDArray[np.float64]
    counts: NDArray[np.int64]

    angle_weighted_probability: NDArray[np.float64]
    angle_weighted_density: NDArray[np.float64]
    center_weighted_density: NDArray[np.float64]
    frame_weighted_density: NDArray[np.float64]

    n_angles: int
    n_candidate_centers: int
    n_accepted_centers: int
    n_contributing_frames: int

    center_atom_indices: NDArray[np.int64]
    frame_indices: NDArray[np.int64]

    cutoff_registry: PairCutoffRegistry
    coordination_filters: tuple[CoordinationCondition, ...]
    averaging: Literal[
        "angle_weighted",
        "center_weighted",
        "frame_weighted",
    ]

    per_frame_counts: NDArray[np.int64] | None = None
    per_frame_probability_density: NDArray[np.float64] | None = None
    per_frame_n_angles: NDArray[np.int64] | None = None
    per_frame_n_accepted_centers: NDArray[np.int64] | None = None
```

```

per_frame_valid: NDArray[np.bool_] | None = None

raw_angles: NDArray[np.float64] | None = None
metadata: dict[str, Any] = field(default_factory=dict)

```

6.1 Histogram fields

`counts[b]` is the number of accepted angles in bin b .

The angle-weighted probability mass is

$$p_b = \frac{\text{counts}_b}{N_{\text{angles}}},$$

and satisfies

$$\sum_b p_b = 1.$$

For bin width $\Delta\theta_b$ in degrees, the probability density is

$$P_b = \frac{p_b}{\Delta\theta_b},$$

with unit degree^{-1} and

$$\sum_b P_b \Delta\theta_b = 1.$$

6.2 Averaging conventions

Angle weighted

Every accepted angle has equal weight:

$$P_{\text{angle}}(\theta) \propto \sum_{\text{angles}} \delta(\theta - \theta_{ijk}).$$

This is the default population distribution. Highly coordinated centers contribute more angles.

Center weighted

Each accepted center first contributes a normalized local histogram. Those histograms are averaged over centers:

$$P_{\text{center}}(\theta) = \frac{1}{N_{\text{centers}}} \sum_j P_j(\theta).$$

Every central atom has equal total weight, regardless of coordination. Centers with fewer than one valid angle do not contribute.

Frame weighted

Each frame first contributes a normalized angle histogram. Valid frame histograms are then averaged:

$$P_{\text{frame}}(\theta) = \frac{1}{N_{\text{valid frames}}} \sum_t P_t(\theta).$$

This is useful for independent ensembles where frames should have equal total weight despite different angle counts.

6.3 distribution property

```
@property
def distribution(self) -> NDArray[np.float64]:
    ...
```

Returns the density selected by averaging. It never discards the other views.

7. High-level algorithm

```
resolve frame and atom selections
|
resolve triplet species
|
coerce cutoff registry and validate required pairs
|
build B-A and B-C neighbor lists per frame
|
compute all filter neighbor unions and counts
|
reject centers failing any condition
|
construct unordered pairs (A == C)
    or Cartesian products (A != C)
|
compute angles from stored minimum-image vectors
|
accumulate angle-, center-, and frame-weighted histograms
|
normalize and construct BondAngleDistributionResult
```

Pseudo-code:

```
for frame in selected_frames:
    neighbors_A = build_neighbor_list(B centers, A candidates, cutoff_AB)
    neighbors_C = build_neighbor_list(B centers, C candidates, cutoff_BC)
    filter_lists = build_lists_required_by_coordination_filters

    frame_hist = zeros(n_bins)

    for center in B centers:
        if not all_coordination_filters_pass:
            continue

        if A == C:
```



```

        endpoint_pairs = unordered_pairs(neighbors_A[center])
    else:
        endpoint_pairs = product(
            neighbors_A[center],
            neighbors_C[center],
        )

    center_angles = angle_from_vectors(endpoint_pairs)
    center_hist = histogram(center_angles)

    raw_counts += center_hist
    frame_hist += center_hist
    center_weighted_accumulator += normalize(center_hist)

frame_weighted_accumulator += normalize(frame_hist)

```

8. Coordination-filter evaluation

For one `CoordinationCondition` with species set S , the module obtains one neighbor list for each $X \in S$ using cutoff r_{BX}^{cut} . For center j , accepted canonical atom indices are combined by set union:

$$\mathcal{N}_S(j) = \bigcup_{X \in S} \mathcal{N}_X(j).$$

The condition passes when

$$N_{\min} \leq |\mathcal{N}_S(j)| \leq N_{\max},$$

using only whichever bounds are defined.

Angle-construction neighborhoods and filter neighborhoods are independent. For example, O-Na-O angles may use a Na-O cutoff while a filter independently counts nearby framework Si and Al atoms.

9. Numerical details

The cosine is computed as

$$c = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|},$$

then clipped:

$$c \leftarrow \min(1, \max(-1, c)).$$

Angles are returned in degrees:

$$\theta = \frac{180}{\pi} \cos^{-1}(c).$$

Zero-length vectors are invalid. Coincident distinct atoms trigger a dedicated error rather than producing undefined angles.

10. Geometric baseline

For two independently and isotropically oriented vectors in three dimensions,

$$P_{\text{random}}(\theta) = \frac{1}{2} \sin \theta.$$

Therefore, a raw probability density has a geometric maximum near 90° even without preferred local geometry. The first module reports the ordinary density because it is standard and directly interpretable. A future extension may provide:

- distributions in $\cos \theta$, with a flat isotropic baseline;
- geometrically corrected angular correlations $P(\theta)/(\frac{1}{2} \sin \theta)$.

These are not part of the first public API.

11. Input constraints and exceptions

Recommended exceptions:

```
class BondAngleError(RuntimeError): ...
class InvalidTripletError(BondAngleError): ...
class MissingPairCutoffError(BondAngleError): ...
class InvalidCoordinationConditionError(BondAngleError): ...
class NoBondAnglesError(BondAngleError): ...
class CoincidentNeighborError(BondAngleError): ...
```

Only cutoffs required by the triplet and coordination filters are validated; unused registry entries do not affect the result.

The function rejects:

- malformed triplets;
- absent species;
- missing $A - B$, $B - C$, or filter cutoffs;
- unsafe cutoffs for any selected frame;
- invalid or empty explicit atom selections;
- nonpositive frame or block strides;
- malformed bins or angle ranges;
- invalid coordination bounds;
- zero-length angle vectors.

The function raises `NoBondAnglesError` if no accepted angle exists over the entire selected collection.

The implementation emits `SparseBondAngleWarning` when:

- fewer than 100 angles contribute;
- fewer than 5% of candidate center-frame observations contribute a valid angle;
- at least one selected frame contains no valid angle;
- some centers pass the coordination filters but have too few endpoint neighbors to form an in-range angle;
- a required RDF-derived cutoff has low feature confidence.

12. Per-frame descriptors and future clustering

When `per_frame=True`, each valid frame has a feature vector

$$\mathbf{h}_t = [P_t(\theta_1), P_t(\theta_2), \dots, P_t(\theta_M)].$$

These descriptors can support future:

- frame clustering;
- rare-event detection;
- diversity selection for MLFF training;
- phase-transition monitoring;
- active-learning triage.

Frames with no valid angle are marked by `per_frame_valid=False`; their stored density row is zero and must not be treated as a normalized distribution.

Per-center histograms and persistent center identifiers are deliberately deferred, but the CSR neighbor API is designed so they can be added later.

13. Complexity and memory

Neighbor search dominates the cost. With N_B centers and candidate endpoint counts N_A and N_C , the transparent blocked search scales in the worst case as

$$O[N_B(N_A + N_C)]$$

per frame.

Angle enumeration adds

$$O\left[\sum_j \binom{q_j}{2}\right]$$

for $A = C$, or

$$O\left[\sum_j q_A(j)q_C(j)\right]$$

for $A \neq C$.

`return_angles=True` stores $O(N_{\text{angles}})$ values. Per-frame histograms store $O(N_f N_{\text{bins}})$ values.

14. Testing requirements

Required analytical tests include:

1. Linear triplet at 180° .
2. Right-angle triplet at 90° .
3. Ideal tetrahedral angle

$$\cos^{-1}(-1/3) \approx 109.471^\circ.$$

4. Periodic-boundary crossing.
5. Triclinic-cell geometry.
6. Symmetric endpoint count $\binom{q}{2}$.
7. Asymmetric endpoint count $q_A q_C$.
8. Exact, range, and combined-species coordination filters.

9. Angle-, center-, and frame-weighted normalization by hand.
10. Identical results for equivalent trajectory, ensemble, and single-frame inputs.
11. Missing-cutoff and no-angle failures.
12. Per-frame descriptor validity masks.

15. Deliberate non-goals

The first implementation does not include:

- joint radial-angular distributions;
- angle autocorrelation or angle lifetimes;
- bond-lifetime-conditioned angles;
- ring-conditioned angle statistics;
- per-center histogram persistence;
- automatic cutoff detection inside `compute_bond_angle_distribution()`;
- Voronoi or nearest-neighbor angle definitions;
- Steinhardt or other bond-orientational order parameters.

16. Example workflows

16.1 Tetrahedral framework angles

```
si_o = PairCutoff.from_rdf_minimum(rdf_si_o)
cutoffs = PairCutoffRegistry.from_cutoffs([si_o])
```

```
result = compute_bond_angle_distribution(
    collection,
    triplet=("O", "Si", "O"),
    cutoffs=cutoffs,
    coordination_filters=[
        CoordinationCondition.exact("O", 4),
    ],
)
```

16.2 Mixed framework bridge

```
cutoffs = PairCutoffRegistry.from_cutoffs(
    [si_o_cutoff, al_o_cutoff]
)
```

```
result = compute_bond_angle_distribution(
    collection,
    triplet=("Si", "O", "Al"),
    cutoffs=cutoffs,
    coordination_filters=[
        CoordinationCondition.exact(("Si", "Al"), 2),
    ],
)
```

16.3 Coordination-conditioned cation geometry

```
result = compute_bond_angle_distribution(
    collection,
    triplet=("O", "Na", "O"),
```

```

    cutoffs=cutoff_registry,
    coordination_filters=[
        CoordinationCondition.exact("O", 6),
        CoordinationCondition.at_most(("Si", "Al"), 2),
    ],
    per_frame=True,
)

```

17. Reproduction checklist

An independent implementation reproduces this module when it:

1. treats the second triplet species as central;
2. obtains all neighborhoods from a fixed cutoff registry;
3. uses the shared minimum-image displacement vectors;
4. counts unordered endpoint pairs when $A = C$;
5. counts Cartesian endpoint products when $A \neq C$;
6. applies combined-species filters using atom-set unions;
7. clips the cosine before `arccos`;
8. retains raw counts and all three documented averaging views;
9. marks frames without valid angles explicitly;
10. rejects missing or unsafe cutoff definitions.

18. Stage S4 execution policy

Endpoint neighborhoods and coordination-filter neighborhoods use the production periodic-neighbor policy without changing their separate scientific meanings. The result stores unified diagnostics in `metadata["neighbor_search"]`. See `docs/specs/analysis/neighbor_search_spec.md`.